

Lambda expressions

Victor Eijkhout, Susan Lindsey

Fall 2026

last formatted: September 15, 2026

1. Why lambda expressions?

Lambda expressions (sometimes incorrectly called 'closures') are 'anonymous functions'. Why are they needed?

- Small functions may be needed; defining them is tedious, would be nice to just write the function recipe in-place.
- C++ can not define a function dynamically, depending on context.

Example:

1. we read `float c`

2. now we want function `float mult(float)` that multiplies by `c`:

```
1 float mult( float x ) { // DOES NOT WORK
2     // multiply x by c
3 };
4 int main() {
5     float c; cin >> c;
6     mult( 5.1f );
7 }
```

2. Introducing: lambda expressions

Traditional function usage:

explicitly define a function and apply it:

```
1 float sum(float x,float y) { return x+y; }  
2 // and in main:  
3 cout << sum( 1.2f, 3.4f );
```

New:

apply the function recipe directly:

Code:

```
1 // lambda/lambdaex.cpp  
2 [] (float x,float y) -> float {  
3     return x+y; } ( 1.5, 2.3 )
```

Output:

```
1 3.8
```

3. Lambda syntax

```
1 [capture] ( inputs ) -> outtype { definition };  
2 [capture] ( inputs ) { definition };
```

- The square brackets are how you recognize a lambda; we will get to the 'capture' later. For now it will often be empty.
- Inputs: like function parameters
- Result type specification `-> outtype`:
can be omitted if compiler can deduce it;
- Definition: function body.

4. Assign lambda expression to variable

Lambda expression assigned to a variable:

Code:

```
1 // lambda/lambdaex.cpp
2 auto summing =
3   [] (float x,float y) -> float {
4     return x+y; };
5 println( "{}",summing( 1.5, 2.3 ) );
6 println( "{}",summing( 3.7, 5.2 ) );
```

Output:

```
1 3.8
2 8.9
```

- This is a variable declaration.
- Uses `auto` for technical reasons; see later.

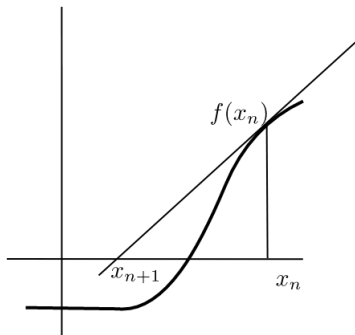
Return type could have been omitted:

```
1 auto summing =
2   [] (float x,float y) { return x+y; };
```

Example of lambda usage: Newton's method

5. Newton's method

$$x_{n+1} = x_n - f(x_n)/f'(x_n)$$



6. Newton for root finding

With

$$f(x) = x^2 - 2$$

zero finding is equivalent to

$$f(x) = 0 \quad \text{for } x = \sqrt{2}$$

so we can compute a square root if we have a zero-finding function.

Newton's method for this f :

$$x_{n+1} = x_n - f(x_n)/f'(x_n) = x_n - \frac{(x_n^2 - 2)}{2x_n} = x_n/2 + 2/x_n$$

Square root computation only takes division!

Exercise 1

Rewrite your code to use lambda functions for f and f_{prime} .

If you use variables for the lambda expressions, put them in the main program.

7. Function pointers

You can pass a function to another function.

In C syntax:

```
1 void f(int i) { /* something with i */ };
2 void apply_to_5( (void)(*f)(int) ) {
3     f(5);
4 }
5 int main() {
6     apply_to_5(f);
7 }
```

8. Lambdas as parameter

We want to write lambda expressions in-line in a function call:

```
1 int main() {  
2     // take a function and apply it to `5`  
3     apply_to_5  
4         ( [] (double x) { println("{} ",x); } );  
5 }
```

9. Lambdas as parameter: the problem

Lambdas have a type that is dynamically generated, so you can not write a function that takes a lambda as argument, because you can't write the type.

```
1 void apply_to_5( /* what? */ func ) {  
2     func(5);  
3 }  
4 int main() {  
5     apply_to_5  
6     ( [] (double x) { cout << x; } );  
7 }
```

10. Lambdas as parameter: the solution

Header:

```
1 #include <functional>
2 using std::function;
```

declare function parameters by their signature
(that is, types of parameters and output):

Code:

```
1 // lambda/lambdaex.cpp
2 void apply_to_5
3     ( function< void(int) > f ) {
4     f(5);
5 }
6     /* ... */
7 apply_to_5
8     ( [] (int i) {
9         println("Int: {}",i);
10    } );
```

Output:

```
1 Int: 5
```

11. Lambdas expressions for Newton

```
1 #include <functional>
2 using std::function;
```

With this, you can declare parameters by their signature (that is, types of parameters and output):

```
1 // newton/newton-lambda.cpp
2 double newton_root
3     ( function< double(double) > f,
4       function< double(double) > fprime ) {
```

This states that f, f_{prime} are in the class of `double(double)` functions: `double` parameter in, `double` result out.

Exercise 2

Rewrite the Newton exercise by implementing a *newton_root* function:

```
1 double root = newton_root( f, fprime );
```

Call the function

1. first with the lambda variables you already created;
2. then directly with the lambda expressions as arguments, that is, without assigning them to variables.

Captures

12. Capture variable

Increment function:

- scalar in, scalar out;
- the increment amount has been fixed through the capture.

Code:

```
1 // lambda/lambdacapture.cpp
2 int n;
3 cin >> n;
4 auto increment_by_n =
5     [n] ( int input ) -> int {
6         return input+n;
7 };
8 println("{} ", increment_by_n (5));
9 println("{} ", increment_by_n (12));
10 println("{} ", increment_by_n (25));
```

Output:

```
1 (input value:
    ↪ 1)
2
3 6
4 13
5 26
```

13. Capture value is copied

Illustrating that the capture variable is copied once and for all:

Code:

```
1 // lambda/lambdacapture.cpp
2 int inc;
3 cin >> inc;
4 auto increment =
5     [inc] ( int input ) -> int {
6         return input+inc;
7     };
8 println("increment by: {}",inc);
9 println("1 -> {}",increment(1));
10 inc = 2*inc;
11 println("1 -> {}",increment(1));
```

Output:

```
1 increment by: 2
2 1 -> 3
3 1 -> 3
```

Exercise 3

Write a program that

- reads a `float` factor;
- defines a function *multiply* of one argument that multiplies its input by that factor.

14. Capture more than one variable

Example: multiply by a fraction.

```
1 int d=2,n=3;  
2 times_fraction = [d,n] (int i) ->int {  
3     return (i*d)/n;  
4 }
```

Exercise 4

- Set two variables

```
1 float low = .5, high = 1.5;
```

- Define a function *is_in_range* of one variable that tests whether that variable is between *low, high*.
(Hint: what is the signature of that function? What is/are input parameter(s) and what is the return result?)

Exercise 5

Extend the Newton exercise to compute roots in a loop:

```
1 // newton/newton-lambda.cpp
2 for ( double n=2.; n<10; n+=.5 ) {
3     cout << "sqrt(" << n << ") = "
4         << newton_root(
5     /* ... */
6         )
7     << '\n';
```

Without lambdas, you would define a function

```
1 double squared_minus_n( double x, double n ) {
2     return x*x-n; }
```

However, the `newton_root` function takes a function of only a real argument. Use a capture to make f dependent on the second parameter.

Exercise 6

Let your Newton calculation record the history of $f(x)$ values. Instead of doing this in the Newton function, use a capture to let the f function record its computed values.

Hint: use a `std::vector<double>`.

15. Derivative by finite difference

You can approximate the derivative of a function f as

$$f'(x) = (f(x+h) - f(x))/h$$

where h is small.

This is called a 'finite difference' approximation.

Exercise 7

Write a version of the root finding function that only takes the objective function:

```
1 double newton_root( function< double(double)> f )
```

and approximates the derivative by a finite difference. You can use a fixed value $h=1e-6$.

Do not reimplement the whole newton method: instead create a lambda expression for the gradient and pass it to the function `newton_root` you coded earlier as second argument.

More lambda topics

16. Capture by value

Normal capture is by value:

Code:

```
1 // lambda/lambdacapture.cpp
2 int n;
3 cin >> n;
4 auto increment_by_n =
5     [n] ( int input ) -> int {
6     return input+n;
7 };
8 println("{} ", increment_by_n (5));
9 println("{} ", increment_by_n (12));
10 println("{} ", increment_by_n (25));
```

Output:

```
1 (input value: 1)
2
3 6
4 13
5 26
```

17. Capture by reference

Capture a variable by reference so that you can update it:

Code:

```
1 // lambda/countif.cpp
2 int count=0;
3 auto count_if_even =
4   [&count] (int i) {
5     if (i%2==0) count++; };
6 for ( int i : {1,2,3,4,5} )
7   count_if_even(i);
8 println("We counted: {}",count);
```

Output:

```
1 We counted: 2
```